GUI-Interaktion og UI-arkitektur

Indhold

Faglige mål 2
Målpind som dagens øvelse dækker 2
Baggrund 3
Event-handling i GUI 3
UI Design Pattern 3
Øvelse 4
1. Event-handling og design mønstre i GUI 4
Hvordan du bliver bedømmet 4

Faglige mål

Målpind som dagens øvelse dækker

1. **Eleven kan foretage GUI-programmering rettet mod både clientside-applikationer, og webbaserede applikationer, der bygger på SPA (Single Page Application)**
2. **Eleven kan implementere en grafisk brugergrænseflade, som understøtter et eller flere funktionelle krav.**
3. **Eleven kan anvende forskellige typer af layout containers.**
4. **Eleven kan udvælge og konfigurere de korrekte kontroller til at understøtte et eller flere funktionelle krav.**
5. ***Eleven kan implementere "event-handling".**
6. ***Eleven kan implementere et UI Design pattern.**
7. *Eleven kan designe, udvikle og anvende "custom-controls".*
8. *Eleven kan konfigurere Application-klassen og dens lifetime-events.*
9. *Eleven kan implementere en multitrådet applikation.*
10. *Eleven kan udføre en simpel usability-test.*

Baggrund

Event-handling i GUI

En grafisk brugergrænseflade (GUI) er interaktiv. Brugeren kan klikke, vælge, skrive, flytte eller trykke på forskellige elementer, og programmet skal reagere på disse handlinger.
Event-handling handler om, hvordan programmet registrerer og reagerer på disse brugerhandlinger.
Et event kan f.eks. være:

- Brugeren klikker på en knap
- Brugeren vælger en værdi i en dropdown
- Brugeren skriver i et tekstfelt
- Brugeren flytter musen
- Brugeren ændrer en indstilling

Når et event opstår, udføres en bestemt event handler, som indeholder den kode, der skal reagere på handlingen.
Det grundlæggende flow
Brugerhandling → Event → Event handler → Ændring i programmet → UI opdateres
Event-handling er dermed en central del af interaktive GUI'er, fordi det forbinder brugerens handlinger med programmets reaktioner.

UI Design Pattern

Et **UI Design Pattern** er en gennemprøvet måde at opbygge en brugergrænseflade på, så bestemte funktioner og informationer præsenteres på en genkendelig og overskuelig måde.

Et pattern beskriver ikke nødvendigvis det præcise design eller den konkrete kode (*all models are wrong princippet*). Det beskriver i stedet en **struktur eller løsning på et typisk problem i brugergrænseflader**.

Eksempler på UI Design Patterns er f.ex: **Master-Detail** – vælg et element og se detaljer som brugt i forgående øvelse hvor en tabel vælges (master) o gen ensartet visning af det valgte tabel (detail).

UI Design Patterns hjælper med at skabe interfaces, der er **overskuelige, genkendelige og nemmere at bruge**.

Når man arbejder med et UI Design Pattern, skal man derfor ikke kun tænke på, **hvordan noget ser ud**, men også på: **Hvordan brugeren interagerer → Hvordan information organiseres → Hvordan UI'et reagerer**

Øvelse

1. Event-handling og design mønstre i GUI

1.1. Du skal implementer mulighed for at opdater dine tabeller når den vises med følgende funktionaliteter:

- Indsæt ny data
- Updater eksisterende data
- Slet eksisterende data

Du bestemmer selv hvordan du vil implementer dette, men din implementation skal opfylde følgende krav:

- Du skal anvend **event** og **event-handling** til opgaven.
- Din implementation må ikke forstyrre det valgte sprog visning implementeret i forgående øvelse.
- Du skal opnå en "ensartet" process for indsæt, opdater og sletning af data for alle vist tabeller ved at anvend en "UI Design mønstre". Her er nogle, eksempler på UI design mønstre blandt mange flere som ikke star på listen som i kan anvende:

UI Design Pattern	Grundidé	Eksempel i jeres projekt
Master-Detail	Vælg noget → se detaljer	Vælg Elever → vis elever
Tabs	Skift mellem relaterede indholdsområder	Faner: Elever / Lærere / Fag
Accordion	Åbn/luk sektioner efter behov	Klik på "Elever" → tabel foldes ud
Navigation Menu	Navigér mellem forskellige funktioner/sider	Home / Elever / Lærere / Fag
Modal Dialog	Vis ekstra information oven på den aktuelle side	Klik på en elev → vis detaljer i dialog
Search/Filter	Begræns eller find data	Søg efter en bestemt elev
Wizard / Stepper	Gennemfør en opgave trin for trin	Opret elev → vælg klasse → bekræft
Card Layout	Præsenter hvert element som et separat kort	Hver elev vises som et kort
Dashboard	Samler flere informationer/funktioner på én side	Statistik over elever, fag og lærere

1.2. **Reflektér over din løsning.**
Du skal kunne vise hvor og hvordan du i din kode har implementeret løsning til event-handling samt UI design mønstre, men også kan forklare hvad begreberne betyder i forbindelse med GUI. Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

Hvordan du bliver bedømmet

- Du skal fremvis kørsel af din application der opfylder de stillede krav.
- Opgave indeholder en "refleksion" øvelse.
    - Du bliver bedømmet via spørgsmåler stillet med udgangspunkt i refleksionen af din øvelser.